



Instituto Politécnico Nacional

Escuela Superior de Cómputo

Selected Topics in Cryptography

Reporte Práctica 04:

“ECDSA”

Alumno:

Sigala Morales Said

2022630413

Grupo: 7CM4

Maestro: Sandra Diaz Santiago

Fecha de realización: 19 de marzo de 2025

Fecha de entrega: 19 de marzo de 2025



PROGRAMMING EXERCISES

Use a file to store the public parameters of the protocol ECDSA: a prime number $p > 255$, the parameters of an elliptic curve $ab \mathbb{Z}_p$, a generator point $G \in E(ab)$ and the order of G , q . These parameters **MUST NOT** be generated at random. Please consider that any user must have access to this file, before generating the key pair or before generating the signature. You are free to define the structure of your file and the format of your file

```
parametros.txt
1  a=17
2  b=28
3  p=257
4  g= 2,29,1
5  d=5
6
```

Esta fue el formato que elegí, los primeros 3 son los parámetros de la curva, el punto g es el punto generador y d es el escalar por el cual se va a multiplicar el generador, aunque no se bien si hacerlo aquí o pedirlo durante la ejecución del código debido a que debe ser un parámetro privado y mantenerlo en los parámetros públicos no resulta lo mas seguro.



Design and implement a function to generate a pair of keys for ECDSA, once that the public parameters were given. The public key must be stored in a text file as a tuple (p, a, b, q, G, B) , where $G \in E(a, b)$.

```
1 def generate_key(a, b, p, d, g):
2
3     x = g[0] % p
4     y = g[1] % p
5     z = g[2] % p
6     G = (x, y, z)
7
8     generador = SumaDoblado.show_generator(a, b, p, G, SumaDoblado.point_add, SumaDoblado.point_double)
9     print(f"El punto G es generador: {generador}")
10
11     puntos_en_curva = SumaDoblado.find_points_on_curve(a, b, p)
12     cardinalidad = len(puntos_en_curva) + 1
13     print(f"Cardinalidad de la curva: {cardinalidad}")
14     if d < 1 or d > cardinalidad:
15         raise ValueError("El valor de d debe estar entre 1 y la cardinalidad de la curva")
16
17     B = SumaDoblado.point_mul(a, b, p, d, G, SumaDoblado.point_add, SumaDoblado.point_double)
18     Kpub = (p, a, b, cardinalidad, G, B)
19     Kpriv = d
20
21     return Kpub, Kpriv
```

Descripción de la función:

Primero se aplica modulo a los valores del punto y genera el nuevo punto. La verificación de que sea una curva no singular y que el primo sea mayor a 255 se hacen en la función que lee los parámetros del punto.

Posterior se llama a la función que muestra si es generador, esto va a devolver si es punto es un generador o si simplemente genera un subgrupo de puntos.

Una vez con eso tenemos que saber cual es el orden de q , este será la cardinalidad del punto, por eso se llama a la función que genera los puntos y luego calcula su cardinalidad sabiendo la longitud del arreglo de los puntos y le suma 1 por el punto al infinito.

Posterior verifica que d sea mayor a 1 y menor al orden q , de no ser así devolverá una excepción.

Ahora con el valor de d comprobado se hace la multiplicación escalar para calcular β . Con esto calculado ponemos generar las claves que es lo que hacemos y las devuelve la función.



Design a toy hash function h , to be used joint with the signature generation for ECDSA. Consider that the output of this toy hash function must be $0 < h(m) < 255$.

```
1 def toy_hash(mensaje: bytes, salt: bytes) -> int:
2     xor_result = bytearray(len(mensaje))
3     for i in range(len(mensaje)):
4         xor_result[i] = mensaje[i] ^ salt[i]
5     final = 0
6     for i, b in enumerate(xor_result):
7         final ^= (b + i) % 256
8     print(f"Hash del mensaje: {final}")
9     return final
```

Descripción de la función:

Básicamente se realiza un xor del mensaje con una salt de la misma longitud del mensaje, se hace xor byte a byte, se tienen dos vectores de bytes y se hace el xor. Esto nos sirve para evitar colisiones que, aunque debido al pequeño espacio de salida es fácil encontrar dos mensajes que produzcan el mismo hash pero intentamos evitar esto implementando una salt.

Posteriormente se hace un xor entre el vector de resultados del procedimiento anterior, antes del xor se suma el valor de la posición a lo que este almacenado en la posición, esto hace que tome relevancia la posición en el texto o sea que "abc" se distinga perfectamente de "cba".



Design and implement a function to do the signature generation for ECDSA. Your function must receive a private key d and a valid message m . Use the toy hash function of point 3. Your function must return the pair (r,s) . When you test your function, consider the public parameters must be already fixed.

```
1 def signature(mensaje: bytes, Kpub: tuple, Kpriv: int, salt: bytes, ke: int) -> tuple:
2     p, a, b, orden, G, B = Kpub
3     d = Kpriv
4     h = toy_hash(mensaje, salt)
5     k = ke % orden
6     if k < 1:
7         raise ValueError("El valor de k debe ser mayor o igual a 1")
8     R = SumaDoblado.point_mul(a, b, p, k, G, SumaDoblado.point_add, SumaDoblado.point_double)
9     r = R[0] % orden
10    if r == 0:
11        raise ValueError("r no puede ser 0")
12    k_inv = pow(k, -1, orden)
13    s = (h + d*r) * k_inv % orden
14    if s == 0:
15        raise ValueError("s no puede ser 0")
16    return r, s
```

Descripción de la función:

Se basa exactamente en el documento dado por la profesora en clase, es básicamente el algoritmo proporcionado donde se calcula un R que es la multiplicación de un escalar entero random entre 1 y orden de $q-1$ por el punto generador. Posterior a ello se toma el valor en 'x' del punto resultado de esa multiplicación escalar y procedemos a calcular s ya con todos los datos proporcionados.



Design and implement a function to do the signature verification for ECDSA.
Your function must read the textfile where the public key is stored and must receive the message m, and the pair (rs) and must return a boolean value: true if the signature is valid or false if it is not valid.

```
1 def verify_from_data(r: int, s: int, h: int, Kpub: tuple) -> bool:
2
3     p, a, b, orden, G, B = Kpub
4     w = pow(s, -1, orden)
5     u1 = h * w % orden
6     u2 = r * w % orden
7
8     P1 = SumaDoblado.point_mul(a, b, p, u1, G,
9                               SumaDoblado.point_add,
10                              SumaDoblado.point_double)
11     P2 = SumaDoblado.point_mul(a, b, p, u2, B,
12                               SumaDoblado.point_add,
13                              SumaDoblado.point_double)
14
15     P = SumaDoblado.point_add(a, b, p, P1, P2)
16     # Comprobamos xP mod orden == r
17     return (P[0] % orden) == r
```

Descripción de la función:

Obtiene los valores de la firma (r,s,h) de un archivo que genera la función firmar, de igual manera se lee de la un .txt la clave publica de quien queramos verificar el mensaje.

Se hace la verificación tal cual viene en el documento dado por la profesora.

Se calcula w el cual es el inverso multiplicativo de "s", luego procedemos a calcular u1 y u2 que como podemos ver ya tenemos todos los datos para ello, luego procedemos a calcular P y para ello dividimos las multiplicaciones por escalar en dos variables y finalmente procedemos a hacer la suma.



EJEMPLO DE APLICACIÓN

```
PS C:\Users\said\Documents\ESCOM\8voSemestre\CRIPTOGRAFIA SELECTED TOPICS IN CRYPTOGRAPHY\Practicas\Practica02> & C:/Users/said/AppData/Local/Microsoft/WindowsApps/python3.12.exe "c:/Users/said/Documents/ESCOM/8voSemestre/CRIPTOGRAFIA SELECTED TOPICS IN CRYPTOGRAPHY/Practicas/Practica02/practica04.py"
Parámetros leídos: a=17, b=28, p=257, d=88, g=(2, 29, 1)
El orden del punto es: 88
El punto G es generador: False
Cardinalidad de la curva: 264
Clave pública: (257, 17, 28, 264, (2, 29, 1), (0, 1, 0))
Clave privada: 88
PS C:\Users\said\Documents\ESCOM\8voSemestre\CRIPTOGRAFIA SELECTED TOPICS IN CRYPTOGRAPHY\Practicas\Practica02> █
```

Generacion de archivos:

```
public_key.txt
1 (761, 2, 1, 773, (531, 556, 1), (617, 95, 1))

signature.txt
1 404
2 121
3 218
```

Como podemos ver en el documento a la derecha tenemos la generación de una clave publica con todos los parámetros que la componen.

En el segundo podemos ver lo que genera la firma, la firma genera 3 valores los cuales son r,s,h respectivamente.

La verificación se realiza con estos archivos, si quisiéramos verificar la firma de alguien lo único que tendríamos que hacer es justo reemplazar estos valores por los que tiene.